

Ein disziplinierter KI-Forschungsassistent

Das Claude Project Template — Wissensarchitektur, Rituale, Grounding

Christian Heinzmann

Hochschule Heilbronn · Promotionskolleg

9. Juni 2026

Agenda

- 1. Motivation: KI in der Promotion
- 2. Zwei Ebenen des Setups
- 3. Ebene 1: Das KI-bewusste Repository
- 4. Ebene 2: Die Drei-Schienen-Wissensarchitektur
- 5. Rituale & asymmetrisches Schreiben
- 6. Schutzmechanismen gegen Halluzination & Entropie
- 7. Die Dr.-Personas
- 8. Konnektoren: MCP (Obsidian + Zotero)
- 9. Kritische Einordnung & Roadmap
- 10. Zusammenfassung

Teil 1

Motivation: KI in der Promotion

Zwei Versagensmodi von LLMs in der Forschung

1. Wissens-Entropie

- Erkenntnisse verstreuen sich über Dutzende Chats.
- Entscheidungen, Begründungen, gelesene Paper gehen verloren.
- Kein kumulatives Gedächtnis über Projekte hinweg.

2. Halluzination

- Erfundene APIs, Flags, Zitate, Fakten.
- In der Wissenschaft **inakzeptabel** — eine falsche Referenz untergräbt alles.

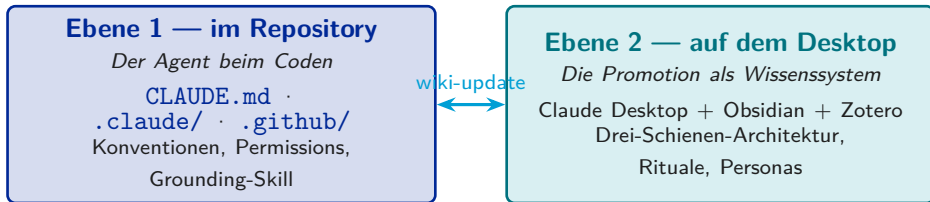
These

Die Antwort ist nicht „ein besseres Modell“, sondern **bessere Struktur und Guardrails**. Das Template macht KI-Nutzung *diszipliniert* — nicht mächtiger, sondern verlässlicher.

Teil 2

Zwei Ebenen des Setups

Zwei komplementäre Ebenen



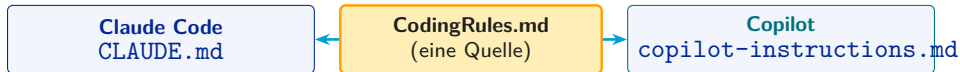
Ebene 1 lebt im austauschbaren Code-Repo (Ausführung). Ebene 2 ist das langlebige Gedächtnis (Strategie + Referenz). Sie koppeln nur an definierten Punkten — nie per Auto-Sync.

Teil 3

Ebene 1: Das KI-bewusste Repository

Ein Regelsatz, zwei Assistenten

- `CLAUDE.md` wird von Claude Code **automatisch geladen**: Projektkontext, Naming-Tabelle, Docstring-Pflicht, Git-Workflow.
- **Single Source of Truth**: `CodingRules.md` wird per `@-Import` in `CLAUDE.md` und in `copilot-instructions.md` eingebunden — Claude & Copilot driften nie auseinander.
- Jedes Claude-Artefakt hat einen **Copilot-Zwilling** (Slash-Command ↔ prompt file), dieselbe Logik in zwei Vendor-Formaten.



Least-Privilege & Grounding

Gestufte Permissions (`settings.json`)

- **auto-erlaubt**: read-only/Dev — `git` `status/diff/log`, `python`, `pip`, `pytest`, `ls`, Read/Grep/Glob.
- **Bestätigung nötig**: `git` `push/reset/rebase`, `rm`, `docker`, `Write`, `Edit`.
- Pro Command tool-scoped: ein Doc-Loader *kann* keine Dateien ändern.

`package-docs`-Skill (anti-hallucination)

- Modell-getriggert über das `description`-Feld (auch bei Fehler-Tracebacks).
- Disziplin: README-Index zuerst, nur 1–2 gezielte Dateien laden, API-Signaturen **wörtlich zitieren**, Quelle als `datei:zeile`.
- Harte Regel: „**Never invent APIs — frag nach**“.

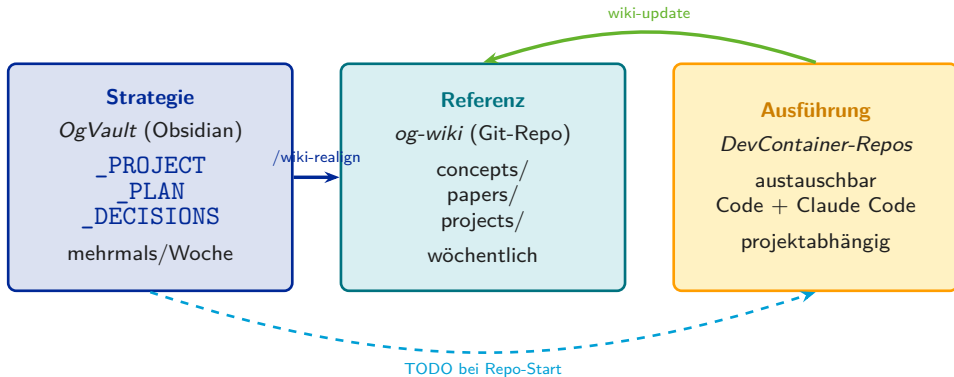
Kernaussage

Antworten werden **geerdet** und copy-paste-sicher.

Teil 4

Ebene 2: Die Drei-Schienen-Wissensarchitektur

Drei Schienen, die nicht auto-synchronisieren



Prinzip: Strategie lebt einmal und entwickelt sich; Referenz wächst und wird nachgeschlagen; Ausführung kommt, macht ihren Job, geht. **Ein Repo darf sterben — das Wissen lebt im Wiki und Vault weiter.**

Teil 5

Rituale & asymmetrisches Schreiben

Jede Kopplung ist ein bewusstes Ritual

- `/push` — Chat → OgVault. Fünf Schritte: Recap → Diff-Plan → Klärung → Schreiben (pro Datei bestätigt) → Session-Closer.
- `wiki-update` — Repo → Wiki. Pro Coding-Session; `.manifest.json` fürs Delta-Tracking; endet mit `git push`.
- `/wiki-realign` — OgVault → Wiki. **Nur** bei Strategiewechsel, **nie** automatisch — Vorschlag, dann Bestätigung.
- **Wiki** → **OgVault** — selten, beim periodischen Review, via `/push`.

Asymmetrie

Lesen ist frei (Auto-Pull, risikolos), **Schreiben nur über das bestätigte `/push`**. So bleibt Brainstorming uninhibiert und der Vault sauber — *Human-in-the-loop write control*.

Teil 6

Schutzmechanismen gegen Halluzination & Entropie

Guardrails, die Vertrauen erzeugen

- **Chat = ephemeres Arbeitsverzeichnis, Vault = persistentes Repository** — bewusste Trennung.
- `_DECISIONS.md` ist **append-only**, nie löschen — auditierbare Spur, warum Pfade verlassen wurden.
- Nie löschen, sondern nach `archive/` verschieben (Scratch ist die einzige Ausnahme).
- **Drift-Check** alle 3–4 Austausche — passives Frühwarnsystem, kein Nagging, verschiebt sich bei laufendem Gedanken.
- **Vertrauenshierarchie** bei `/sync`: Zotero = Master für Metadaten, Obsidian = Master für Notizen.
- **Nie Zitate erfinden** — explizite Regel im System-Prompt.

Zweisprachig nach Design: **Konversation Deutsch**, aber **aller akademischer Inhalt Englisch** (RQs, Titel, Definitionen, Code) — und LaTeX immer als `.tex`-Quelle mit Versionsmarken.

Teil 7

Die Dr.-Personas

Modulare Rollen-Prompts

- Orthogonale, ladbare Session-Modi (**eine pro Session**), per natürlichem Trigger „Lade Dr. X“.

Dr. EveryDay	Verifizierte Alltags-/Technikfragen: offizielle Quellen vor Aggregatoren, harte Anti-Halluzination, „nicht verifiziert“-Markierung.
Dr. Code	Vollständige, dokumentierte Dateien + README, Doxygen-Docstrings, Typ-Hints, Versionschecks.
Dr. Analyse	Psychologische Text-/E-Mail-Optimierung aus vier Leserperspektiven (Empfänger, Autor, PR, Worst-Case).
Dr. Mail	Schlanke formale E-Mail-Korrektur — Grammatik/Stil, ohne den Inhalt zu verändern.

Persistiert über drei Claude-*Memory-Edits* (Workflow, Vault-Pfad, Persona-Trigger) — überlebt Sessions ohne erneutes Einfügen. Teilbar mit Studierenden und Kolleg:innen.

Teil 8

Konnektoren: MCP (Obsidian + Zotero)

Obsidian — zwei Varianten

- **A: Filesystem-MCP** (Default) — kein Plugin, reines Markdown, Obsidian muss nicht laufen.
- **B: Local REST API** — Dataview/Backlinks/Tags, Ports 27124/27123, Obsidian muss laufen.

Zotero

- `ZOTERO_LOCAL=true` — lokal, Embeddings via all-MiniLM-L6-v2: **keine Daten verlassen die Maschine.**

Dokumentierte Footguns

- `zotero-mcp-server` (von 54yyyu) installieren — **nicht** `zotero-mcp`; beide existieren und liefern dieselbe CLI, der falsche scheitert *stumm*.
- MSIX-Config liegt unter `%LOCALAPPDATA%\...` — das Auto-Setup schreibt in den falschen klassischen Pfad.

Kernaussage

Voraussetzung: mind. Claude Pro; MCP nur Desktop/Web, nicht mobil.

Teil 9

Kritische Einordnung & Roadmap

Stärken

- Struktur gegen Wissens-Entropie
- Grounding gegen Halluzination
- Human-in-the-loop-Schreiben
- Vendor-agnostisch (Claude + Copilot)
- Lokal & datensparsam

Offene Punkte (Roadmap)

- **Daten-/KI-Governance** explizit dokumentieren: was verlässt die Maschine, GDPR, no-training-Plan.
- **Keine Evaluation**: ein gemessener „grounded vs. ungrounded“-Vergleich macht aus der Design-Behauptung Evidenz.
- **Naming-/Doku-Drift**: `_TEMPLATES` vs. `_PROJECT_TEMPLATES`; `src/NewWandB` vs. `src/Template`.
- **Duplikation**: `read-package-a/b` + Copilot-Zwillinge = 6 Dateien für 3 Logiken → parametrisieren.
- **Aufräumen**: `.claude/worktrees/` & `temp.md`

Teil 10

Zusammenfassung

- Nicht „mehr KI“, sondern **disziplinierte KI**: Struktur + Rituale + Grounding.
- **Zwei Ebenen**: der Agent im Repo (Ausführung) und das Wissenssystem auf dem Desktop (Strategie + Referenz).
- **Schreiben ist ein bewusster, bestätigter Akt** — der Vault bleibt vertrauenswürdig, der Chat bleibt frei.
- Roadmap: *Governance-Notiz + kleine Eval + Konsolidierung der Naming-/Doku-Drift.*

Fragen & Diskussion

Zusammen mit Präsentation 1 (DevContainer) ergibt sich die volle Forschungs-Infrastruktur.